Software Engineering Lab - Project-Based Learning (PBL)

Smart Classroom Electricity Management System

Vacancy detection, AC-state inference and appliance control

Individual Project Report

Particular	Details
Name	Vineet Shah
Class / Batch	5S CSE / A6 - B2
Roll No.	28
Course	Software Engineering Lab (24CS01PR0502)
Academic year	2026-27
Implemented modules	Module 1: Vacancy and AC-State Detection Engine Module 2: Monitoring Dashboard and Control
Project management	Jira (epics, user stories, tasks, sub-tasks, sprints, bugs)
Date	25-10-2026

1. Project Title and Problem Statement

Title: Smart Classroom Electricity Management System

Lights, fans and air conditioners stay on in classrooms and laboratories after people leave. Manual switching and fixed timers cannot follow irregular lecture schedules, which raises utility bills and wastes energy. PIR sensors cannot recognise appliances and miss people who sit still, and a single temperature sensor cannot tell whether the AC is still running.

The proposed system watches the room with a CCTV camera and a YOLOv8 detector (people, fans and lights), measures the temperature of four room regions with DHT11 sensors, confirms that the room is vacant, raises alerts for appliances left on, infers whether the AC is still running from the temperature drop (Delta T) across the four regions, and lets an operator switch fans and lights through an ESP32 relay controller from a web dashboard.

The problem statement comes from my Idea Lab project "Electricity Management through Computer Vision & Temperature Sensing". For this individual SE PBL, two software modules of that system were selected, implemented, managed in Jira and tested.

2. System and Module Description

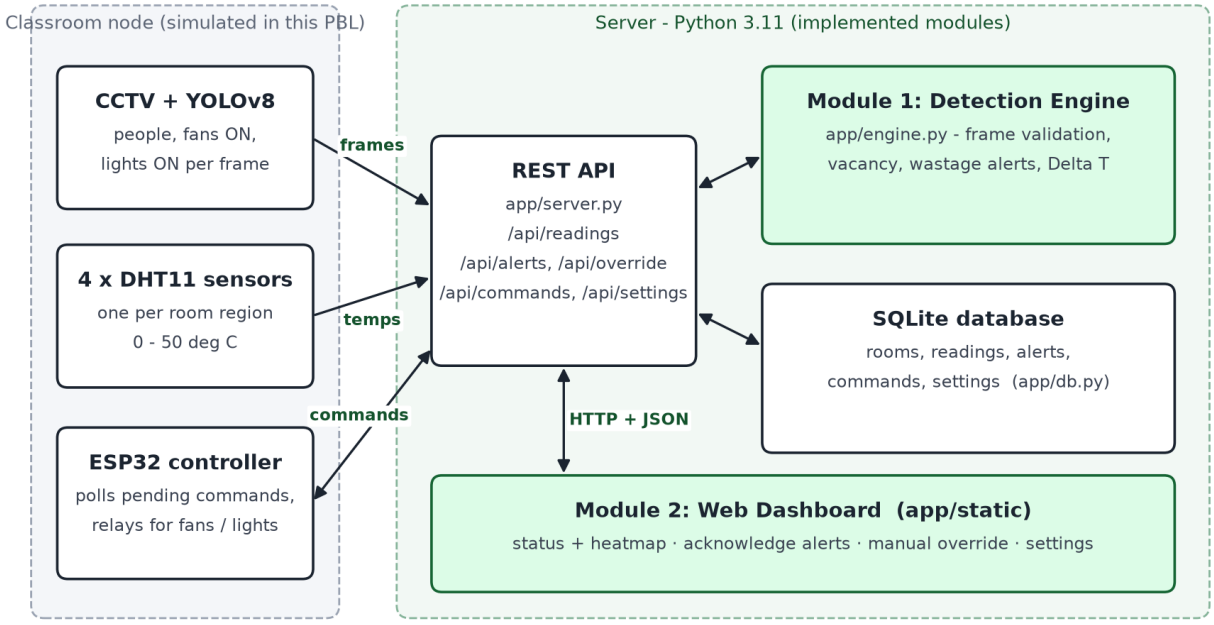


Figure 1: Architecture of the prototype (green boxes are the two implemented modules)

Part	Role in this prototype
Module 1 - Vacancy and AC-State Detection Engine (app/engine.py)	Validates each frame, confirms vacancy over consecutive empty frames, raises fan/light wastage alerts and infers the AC state from the 4-region Delta T.
Module 2 - Monitoring Dashboard and Control (app/server.py, app/db.py, app/static)	REST API and SQLite database; web dashboard with room status, heatmap, alert acknowledgement, manual relay override and detection settings.
Classroom node (app/simulator.py)	The CCTV + YOLOv8 detector, the four DHT11 sensors and the ESP32 controller were not available for this prototype. The simulator sends the same frames to the API and executes relay commands exactly as the ESP32 would.

Item	Technology
Language / runtime	Python 3.11 (standard library only: http.server, sqlite3, json, unittest)
Database	SQLite (tables: rooms, readings, alerts, commands, settings)
User interface	HTML, CSS, JavaScript (fetch API), served by the Python server
Testing	Python test runner and unittest; dashboard UI tests in headless Microsoft Edge
Project management	Jira - backlog, sprints, board, bug tracking

3. Jira Project

Item	Details
Jira space (project)	Smart Classroom Electricity Management - key SCEM (Scrum, team-managed)
Site	rbunagpur-team-zhx8jd75.atlassian.net
Work items	4 epics, 9 user stories, 10 tasks, 18 subtasks, 4 bugs = 45
Hierarchy	Epic -> Story / Task / Bug -> Subtask (SCEM-1 to SCEM-4 are the epics)
Workflow	To Do -> In Progress -> Done
Status on 26-10-2026	43 Done, 2 In Progress (SCEM-23 report and presentation, and its epic SCEM-4)

The backlog was written in jira/backlog.py and imported into Jira with the CSV importer (jira/jira_import.csv). Work items were then planned into three sprints and moved through the workflow as the work was completed; each bug was moved to In Progress, given a fix and retest comment, and closed.

Epic	Jira key	Description	Label
Module 1: Vacancy and AC-State Detection Engine	SCEM-1	Edge decision engine: validates camera/sensor frames, confirms vacancy over consecutive frames, raises fan/light wastage alerts and infers the AC state from the 4-region temperature drop (FR-1.1 to FR-1.8).	module-1
Module 2: Monitoring Dashboard and Control	SCEM-2	REST API + SQLite database + web dashboard: room status and heatmap, alert acknowledgement, manual ESP32 relay override and detection settings (FR-2.1 to FR-2.7).	module-2
Testing and Quality Assurance	SCEM-3	Test case design (EP, BVA, validation), unit, integration and UI test automation, test execution, bug tracking and retesting.	testing
Project Setup and Documentation	SCEM-4	Requirements, project structure, simulator for the hardware, report and presentation.	docs

Sprint and goal	Dates	Work items	Points	State
SCEM Sprint 1 Detection engine (Module 1) and requirements	25-10-2026 19:30 - 19:50	SCEM-5, SCEM-6, SCEM-7, SCEM-8, SCEM-9, SCEM-10	16	Completed
SCEM Sprint 2 Dashboard, API, database and simulator (Module 2)	25-10-2026 19:50 - 20:00	SCEM-11, SCEM-12, SCEM-13, SCEM-14, SCEM-15, SCEM-16	22	Completed
SCEM Sprint 3 Testing, bug fixing and documentation	25-10-2026 20:00 - 01-11-2026	SCEM-17, SCEM-18, SCEM-19, SCEM-20, SCEM-21, SCEM-22, SCEM-23, SCEM-24 to SCEM-27 (bugs)	16	Active

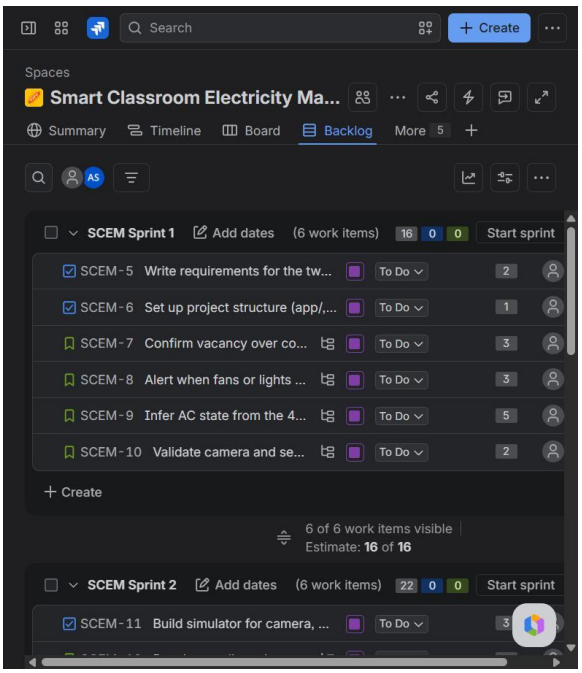


Figure 2: Sprint planning - backlog items placed in SCEM Sprint 1 and Sprint 2

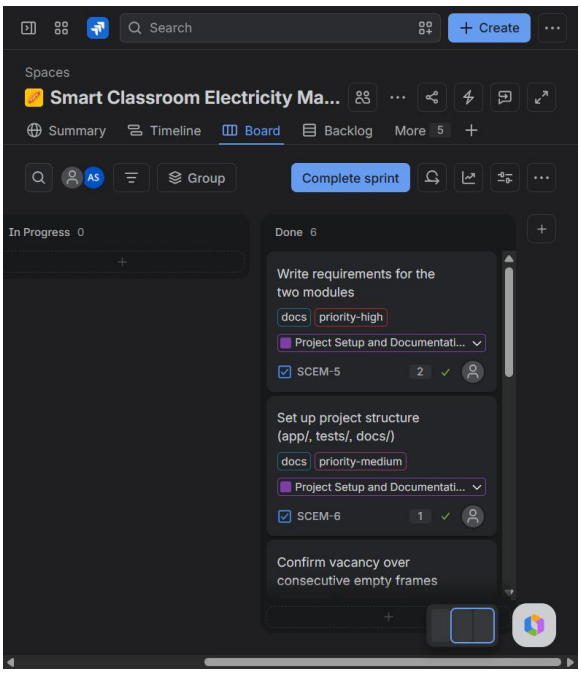


Figure 3: SCEM Sprint 1 board - all work items moved to Done

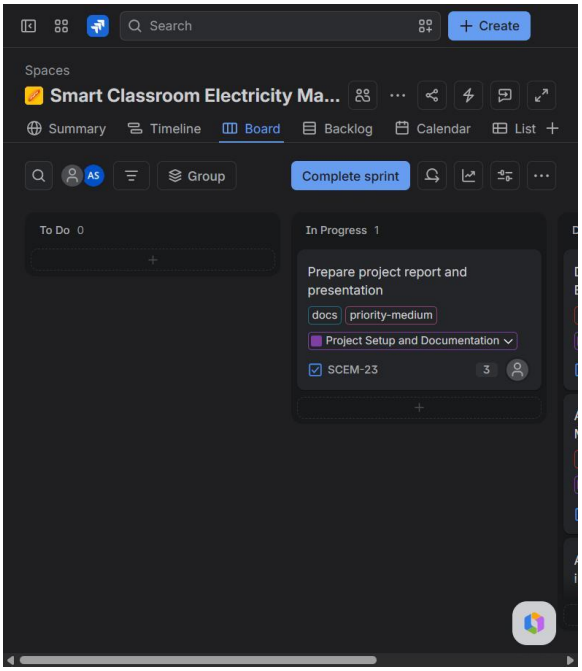


Figure 4: Active SCEM Sprint 3 board - report task In Progress

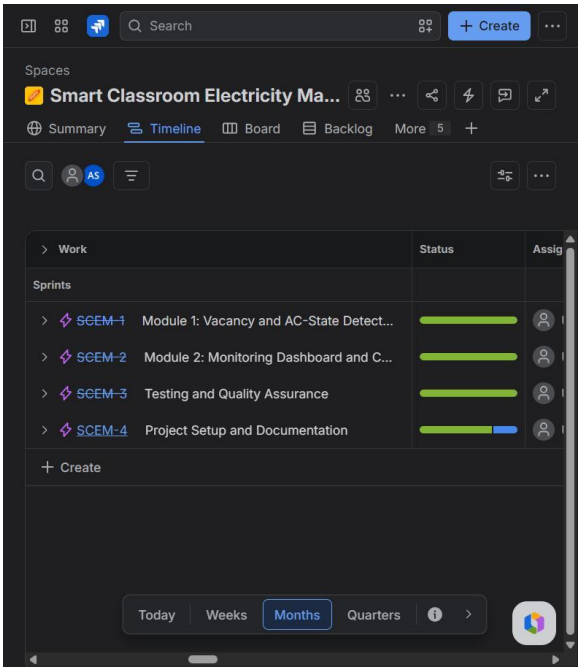


Figure 5: Timeline - progress of the four epics

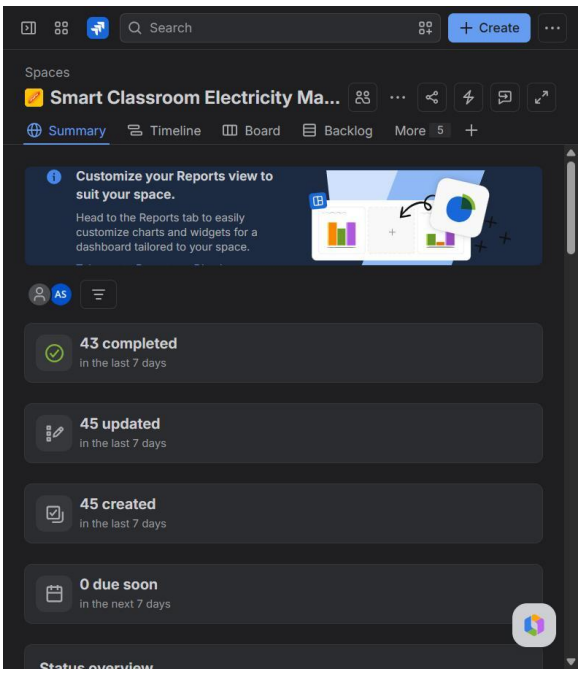


Figure 6: Project summary - 45 work items created, 43 completed

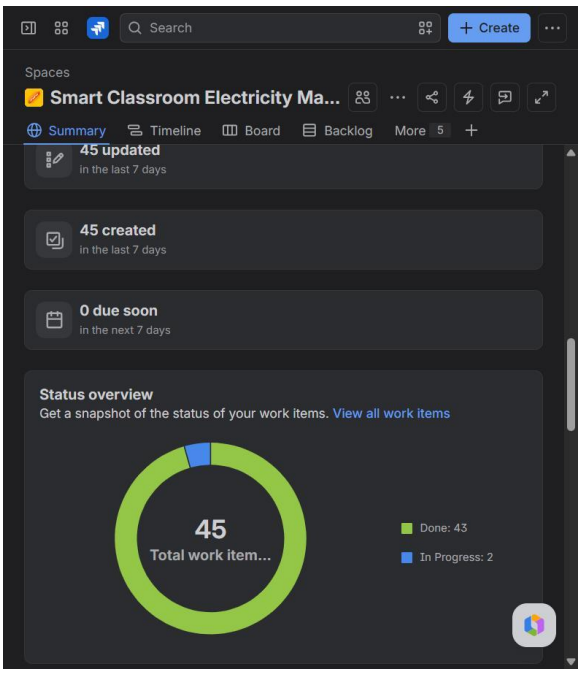


Figure 7: Status overview - Done 43, In Progress 2

4. Product Backlog

Jira key	Type	Summary	Epic	Priority	Points	Sprint
SCEM-5	Task	Write requirements for the two modules	SCEM-4	High	2	S1
SCEM-6	Task	Set up project structure (app/, tests/, docs/)	SCEM-4	Medium	1	S1
SCEM-7	Story	Confirm vacancy over consecutive empty frames	SCEM-1	High	3	S1
SCEM-8	Story	Alert when fans or lights are left ON in a vacant room	SCEM-1	High	3	S1
SCEM-9	Story	Infer AC state from the 4-region temperature drop	SCEM-1	Highest	5	S1
SCEM-10	Story	Validate camera and sensor frames	SCEM-1	Medium	2	S1
SCEM-11	Task	Build simulator for camera, DHT11 sensors and ESP32	SCEM-4	Medium	3	S2
SCEM-12	Story	Receive readings through a REST API and store them	SCEM-2	High	3	S2
SCEM-13	Story	Show room status and region heatmap on the dashboard	SCEM-2	High	5	S2
SCEM-14	Story	List and acknowledge open alerts	SCEM-2	High	3	S2
SCEM-15	Story	Manual fan/light override through the ESP32	SCEM-2	High	5	S2
SCEM-16	Story	Edit detection settings from the dashboard	SCEM-2	Medium	3	S2
SCEM-17	Task	Design test cases (EP, BVA, validation, integration)	SCEM-3	High	3	S3
SCEM-18	Task	Automate unit tests for Module 1	SCEM-3	High	2	S3
SCEM-19	Task	Automate API and integration tests	SCEM-3	High	3	S3
SCEM-20	Task	Automate dashboard UI tests in Microsoft Edge	SCEM-3	Medium	3	S3
SCEM-21	Task	Execute test run 1 and log defects in Jira	SCEM-3	High	1	S3
SCEM-22	Task	Retest after bug fixes (run 2)	SCEM-3	High	1	S3
SCEM-23	Task	Prepare project report and presentation	SCEM-4	Medium	3	S3

Sub-tasks (18): US1 - Implement vacancy counter in RoomMonitor; US1 - Unit tests TC_M1_11 to TC_M1_13; US2 - Implement fan/light wastage alerts; US2 - Unit tests TC_M1_14 to TC_M1_18; US3 - Implement Delta T calculation and AC inference; US3 - Unit tests TC_M1_19 to TC_M1_26; US4 - Implement Frame.from_dict validation; US4 - Unit tests TC_M1_01 to TC_M1_10; US5 - Create SQLite schema (rooms, readings, alerts, commands, settings); US5 - Implement POST /api/readings; US6 - Build status card and heatmap UI; US6 - UI tests TC_UI_01 to TC_UI_04; US7 - Implement alert list and acknowledge API; US7 - Build acknowledge form; US8 - Implement override and command API; US8 - Build override form and command list; US9 - Implement settings API with persistence; US9 - Build settings form.

5. User Stories

Key	User story and acceptance criteria	Points
US1	Confirm vacancy over consecutive empty frames As a facility manager I want a room to be marked vacant only after N consecutive frames without people, so that short absences do not raise alerts. Acceptance: FR-1.3, FR-1.4; settings limit FR-1.8.	3
US2	Alert when fans or lights are left ON in a vacant room As a facility manager I want an alert when a fan or light stays ON after everyone leaves. Acceptance: FR-1.5 (once per occurrence, again if switched off and on).	3
US3	Infer AC state from the 4-region temperature drop As a facility manager I want to know if the AC is still running in a vacant room without a power meter. Acceptance: FR-1.6 (mean drop >= threshold after wait_seconds), FR-1.7 (cancel if someone enters).	5
US4	Validate camera and sensor frames As the system I want invalid frames rejected with a clear message. Acceptance: FR-1.1, FR-1.2.	2
US5	Receive readings through a REST API and store them As the sensor node I want to post readings to /api/readings so that they are processed and stored. Acceptance: FR-2.1.	3
US6	Show room status and region heatmap on the dashboard As an operator I want to see the state, counts, AC check and a 2x2 temperature heatmap of a room. Acceptance: FR-2.2.	5
US7	List and acknowledge open alerts As an operator I want to acknowledge alerts with my name and remarks. Acceptance: FR-2.3.	3
US8	Manual fan/light override through the ESP32 As an operator I want to switch fans or lights ON/OFF from the dashboard. Acceptance: FR-2.4, FR-2.5.	5
US9	Edit detection settings from the dashboard As an administrator I want to change the vacancy, wait and threshold settings and keep them after a restart. Acceptance: FR-2.6, FR-2.7.	3

6. Implementation of the Two Modules

6.1 Module 1: Vacancy and AC-State Detection Engine

The engine (app/engine.py, class RoomMonitor) receives one frame at a time. A frame carries the YOLOv8 counts (people, fans ON, lights ON) and the four DHT11 temperatures. The room becomes vacant only after N consecutive empty frames; the four temperatures are then recorded and, after wait_seconds, compared with the current ones. A mean drop of at least delta_t_threshold means the AC is still cooling an empty room.

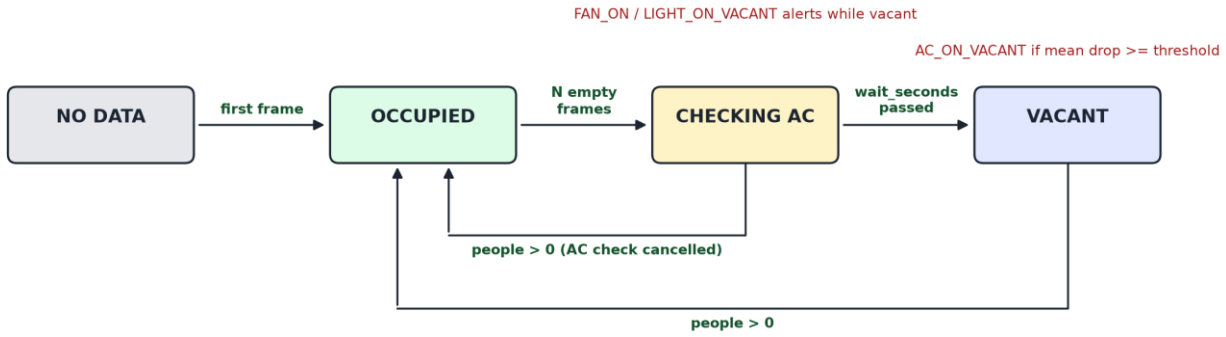


Figure 8: Room states maintained by Module 1

ID	Functional requirement (Module 1)
FR-1.1	Accept frames: timestamp, people, fans_on, lights_on and exactly 4 region temperatures.
FR-1.2	Input validation: counts are whole numbers 0-50, temperatures 0-50 deg C (DHT11 range), timestamps strictly increasing.
FR-1.3	Vacancy is confirmed after at least vacancy_frames consecutive frames with people = 0; any person resets the count.
FR-1.4	On vacancy the four temperatures are recorded and the state becomes CHECKING_AC.
FR-1.5	FAN_ON_VACANT / LIGHT_ON_VACANT alerts while vacant, once per occurrence.
FR-1.6	After wait_seconds: mean drop >= delta_t_threshold -> AC inferred ON and AC_ON_VACANT.
FR-1.7	A person entering during the check cancels it; the room is OCCUPIED again.
FR-1.8	Settings: vacancy_frames 1-300, wait_seconds 30-600, delta_t_threshold 0.1-5.0.

Core of the engine - RoomMonitor.process() (complete file in Appendix A):

```
    def process(self, frame):
        if self.last_timestamp is not None and frame.timestamp <= self.last_timestamp:
            raise ValidationError("timestamp must be later than the previous frame.")
        self.last_timestamp = frame.timestamp
        self.last_frame = frame
        alerts = []

        if frame.people > 0:
            self._reset_to_occupied()
            return alerts

        self.empty_frames += 1
        if self.state == OCCUPIED and self.empty_frames >= self.settings.vacancy_frames:
            self.state = CHECKING_AC
            self.initial_temps = list(frame.temps)
            self.check_started = frame.timestamp

        if self.state == OCCUPIED:
            return alerts

        alerts += self._appliance_alerts(frame)

        if self.state == CHECKING_AC and frame.timestamp - self.check_started >=
self.settings.wait_seconds:
            drops = [start - end for start, end in zip(self.initial_temps, frame.temps)]
            # Compare at full precision (6 decimals only removes float noise); rounding to 2 decimals
            # before comparing turned a 0.499 deg C drop into 0.50 and raised a false AC alert (BUG-02).
            mean_drop = round(sum(drops) / REGIONS, 6)
            self.last_mean_drop = round(mean_drop, 3)
            self.ac_inferred_on = mean_drop >= self.settings.delta_t_threshold
            self.state = VACANT
            if self.ac_inferred_on:
                alerts.append(self._alert(AC_ON_VACANT, frame.timestamp,
                                          "AC appears to be ON in vacant room %s (mean temperature drop
%.2f deg C "
                                          "in %d s)." % (self.room_id, self.last_mean_drop,
self.settings.wait_seconds),
                                          self.last_mean_drop))
        return alerts
```

6.2 Module 2: Monitoring Dashboard and Control

The server (app/server.py) exposes a REST API, passes readings to Module 1, stores readings, alerts, relay commands and settings in SQLite (app/db.py) and serves the dashboard (app/static). The dashboard refreshes every 2 seconds.

Method and path	Purpose	Validation / errors
POST /api/readings	Frame from the classroom node -> Module 1 -> stored with any alerts	400 room_id is required / frame errors (FR-1.2); 404 unknown room; 400 invalid JSON
GET /api/rooms, /api/rooms/<id>	Room details, engine status and open alert count	404 unknown room
GET /api/alerts?room=&status=	Alerts, newest first	-
POST /api/alerts/<id>/ack	Acknowledge an alert	operator 3-40 letters; remarks <= 200; 404; 409
POST /api/override	Queue a fan/light ON/OFF command for the ESP32	appliance fan/light; action ON/OFF; operator; reason 5-100
GET /api/commands, POST /api/commands/<id>/done	ESP32 reads and completes commands	404; 409
GET / PUT /api/settings	Detection settings, stored in the database	limits of FR-1.8

Table	Columns
rooms	id, name, fans, lights, acs
readings	id, room_id, ts, people, fans_on, lights_on, t1, t2, t3, t4, state
alerts	id, room_id, ts, type, message, value, status, ack_by, ack_remarks, ack_at
commands	id, room_id, appliance, action, operator, reason, status, created_at, done_at
settings	key, value

ID	Functional requirement (Module 2)
FR-2.1	Receive readings through the API, pass them to Module 1, store readings and alerts.
FR-2.2	Show state, counts, AC check, last reading and a 2x2 heatmap; NO DATA for rooms without readings.
FR-2.3	List open alerts and acknowledge one (operator name and optional remarks).
FR-2.4	Manual override of fans/lights with operator name and reason.
FR-2.5	ESP32 picks up pending commands and marks them done.
FR-2.6	Edit settings on the dashboard; stored, applied at once, kept after restart.
FR-2.7	Dashboard validation messages use the field labels shown on the form.

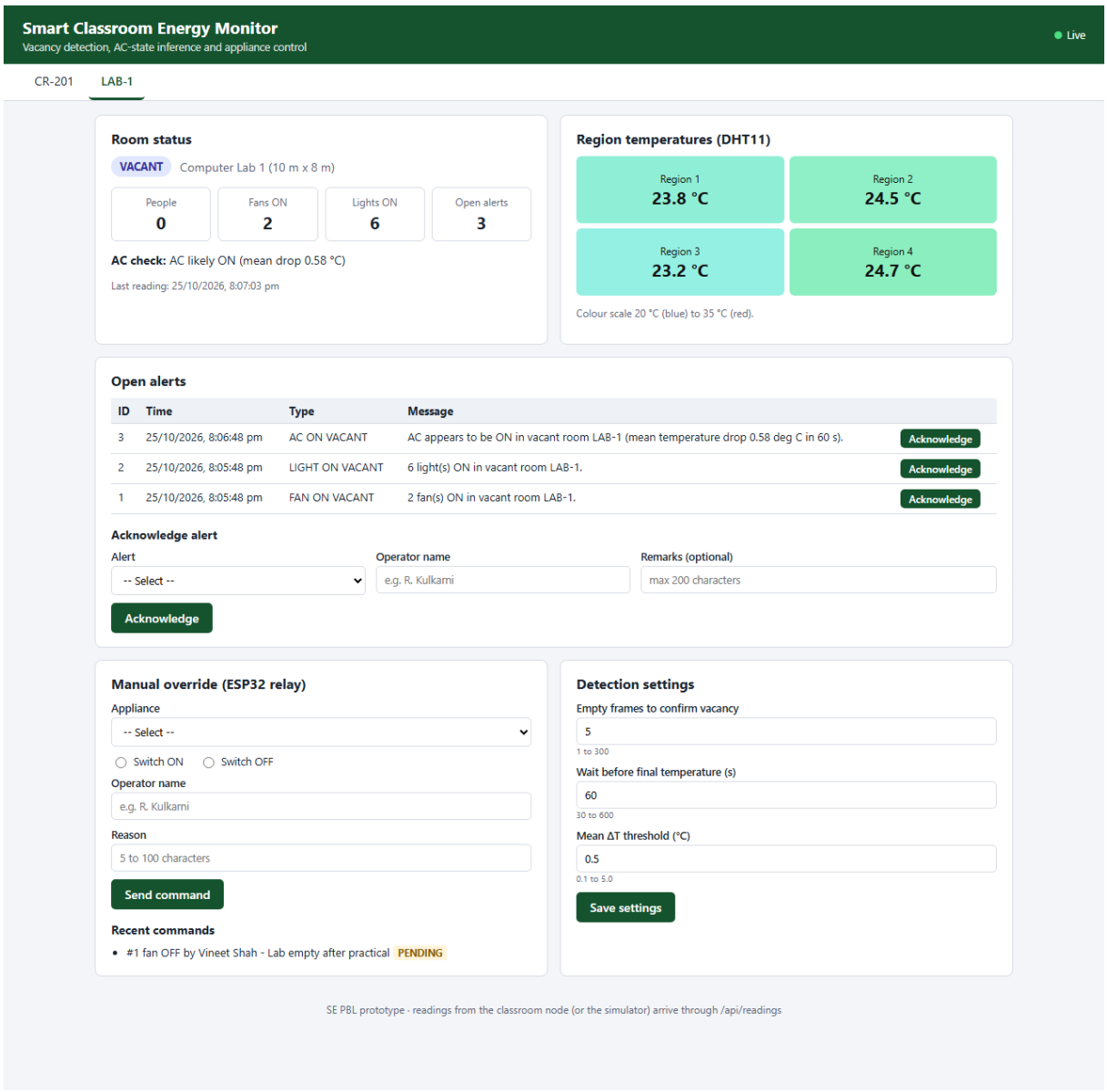


Figure 9: Dashboard - LAB-1 after the class left with 2 fans, 6 lights and the AC on (simulator run on 25-10-2026); a fan OFF command is pending for the ESP32

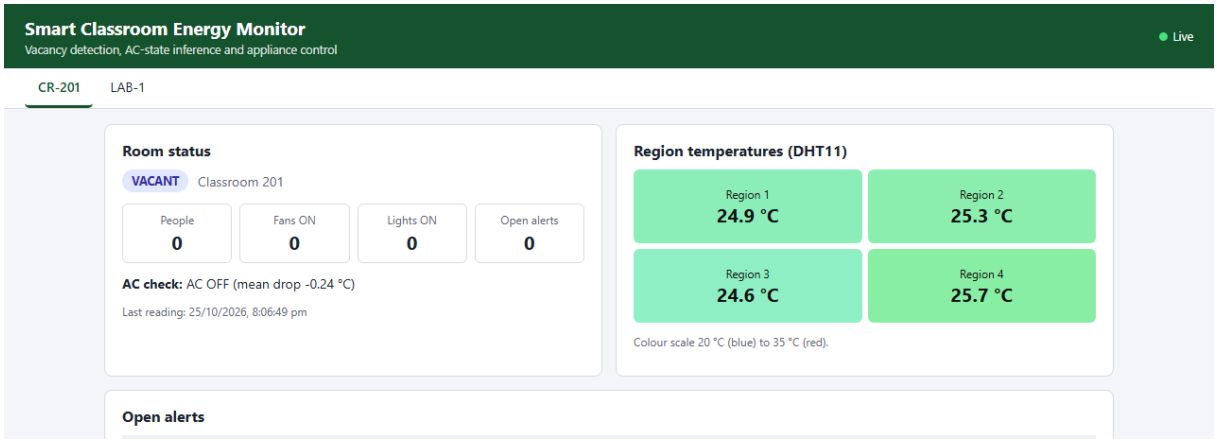


Figure 10: Dashboard - CR-201 after everything was switched off: AC inferred OFF (room warmed by 0.24 deg C)

Integration: readings flow from the classroom node through the API into Module 1; the alerts it returns are written to the database and shown by the dashboard. Commands sent from the dashboard are stored as

PENDING, executed by the ESP32 (simulated) and marked DONE; the next readings then show the appliance switched off. Settings saved on the dashboard are applied to the running engine.

7. Source Code

File	Lines	Contents
app/engine.py	187	Module 1 - frame validation, RoomMonitor state machine, settings
app/server.py	252	Module 2 - REST API, request validation, static file server
app/db.py	147	Module 2 - SQLite schema and data access
app/simulator.py	82	Simulated camera + DHT11 + ESP32 node
app/static/index.html	103	Dashboard page
app/static/app.js	205	Dashboard logic (polling, forms)
app/static/style.css	68	Dashboard styles
tests/helpers.py	87	Test helpers: frame builders, throw-away server, HTTP client
tests/engine_cases.py	186	32 Module 1 test cases
tests/api_cases.py	269	24 Module 2 API cases and 7 integration cases
tests/ui/ui_cases.js	136	15 dashboard UI test cases
tests/run_tests.py	80	Executes all cases and records actual results
tests/test_unit.py	29	unittest entry point
jira/backlog.py	161	Jira backlog and CSV export

```
python -m app.server --port 8090 --db data/energy.db        # start server + dashboard
python -m app.simulator --room LAB-1 --scenario ac_left_on  # simulated classroom node
python -m tests.run_tests run2-retest                        # execute all 78 test cases
python -m unittest -v tests.test_unit                        # unit + API + integration via unittest
```

8. Test Cases and Test Results

Technique	How it was applied
Unit testing	Module 1 engine methods tested directly (TC_M1_xx)
Functional testing	Every functional requirement checked through its output
Integration testing	Engine + API + database + ESP32 command loop (TC_INT_xx, TC_UI_08)
Boundary Value Analysis	min-1 / min / max / max+1 for temperatures, counts, frames, wait, threshold, name, remarks and reason lengths
Equivalence Partitioning	valid and invalid classes for every input field
Validation testing	every error message of the API and of the dashboard forms

Test cases were designed from the requirements (docs/requirements.md) before execution. They are automated: Python cases call the engine or the running server; UI cases fill the dashboard forms in Microsoft Edge and read the displayed result. Actual results below are from the final run (run2-retest, 2026-10-25 20:04:31); the first run is compared in Section 10.

M1 Detection Engine

Test Case ID	Module	Test Scenario	Test Data	Expected Result	Actual Result	Status
TC_M1_01	M1 Detection Engine	Valid frame from camera and sensors [Validation / EP (valid)]	people=3, fans_on=2, lights_on=4, temps=[24.5, 25.0, 24.8, 25.2]	accepted	accepted	**PASS**
TC_M1_02	M1 Detection Engine	Only 3 region temperatures sent [Validation / EP (invalid)]	temps=[24.5, 25.0, 24.8]	temps must list exactly 4 region temperatures.	temps must list exactly 4 region temperatures.	**PASS**
TC_M1_03	M1 Detection Engine	Region temperature at DHT11 upper limit [BVA (max)]	temps=[24.5, 50.0, 24.8, 25.2]	accepted	accepted	**PASS**
TC_M1_04	M1 Detection Engine	Region temperature above DHT11 range [BVA (max + 0.1)]	temps=[24.5, 50.1, 24.8, 25.2]	temperature of region 2 must be between 0 and 50 deg C.	temperature of region 2 must be between 0 and 50 deg C.	**PASS**
TC_M1_05	M1 Detection Engine	Region temperature at DHT11 lower limit [BVA (min)]	temps=[0.0, 25.0, 24.8, 25.2]	accepted	accepted	**PASS**
TC_M1_06	M1 Detection Engine	Region temperature below DHT11 range [BVA (min - 0.1)]	temps=[-0.1, 25.0, 24.8, 25.2]	temperature of region 1 must be between 0 and 50 deg C.	temperature of region 1 must be between 0 and 50 deg C.	**PASS**
TC_M1_07	M1 Detection Engine	Negative people count [EP (invalid)]	people=-1	people must be between 0 and 50.	people must be between 0 and 50.	**PASS**
TC_M1_08	M1 Detection Engine	People count sent as text [EP (invalid)]	people="3"	people must be a whole number.	people must be a whole number.	**PASS**
TC_M1_09	M1 Detection Engine	Fractional fan count [EP (invalid)]	fans_on=1.5	fans_on must be a whole number.	fans_on must be a whole number.	**PASS**
TC_M1_10	M1 Detection Engine	Frame with the same timestamp as the previous frame [Validation]	frames at t=10 and t=10	timestamp must be later than the previous frame.	timestamp must be later than the previous frame.	**PASS**
TC_M1_11	M1 Detection Engine	4 consecutive empty frames [BVA (frames - 1)]	vacancy_frames=5; 2 occupied + 4 empty frames	state=OCCUPIED, alerts=[]	state=OCCUPIED, alerts=[]	**PASS**

Test Case ID	Module	Test Scenario	Test Data	Expected Result	Actual Result	Status
TC_M1_12	M1 Detection Engine	5 consecutive empty frames confirm vacancy [BVA (frames)]	vacancy_frames=5; 2 occupied + 5 empty frames	state=CHECKING_AC, alerts=[]	state=CHECKING_AC, alerts=[]	PASS
TC_M1_13	M1 Detection Engine	Person re-enters after 4 empty frames [Unit (path)]	4 empty, 1 frame with people=1, 4 empty	state=OCCUPIED, alerts=[]	state=OCCUPIED, alerts=[]	PASS
TC_M1_14	M1 Detection Engine	Fans left ON after everyone leaves [Functional / EP]	5 empty frames + 5 more with fans_on=2	state=CHECKING_AC, alerts=[FAN_ON_VACANT]	state=CHECKING_AC, alerts=[FAN_ON_VACANT]	PASS
TC_M1_15	M1 Detection Engine	Lights left ON after everyone leaves [Functional / EP]	10 empty frames with lights_on=6	state=CHECKING_AC, alerts=[LIGHT_ON_VACANT]	state=CHECKING_AC, alerts=[LIGHT_ON_VACANT]	PASS
TC_M1_16	M1 Detection Engine	Fans and lights switched off when leaving [Functional / EP]	10 empty frames, fans_on=0, lights_on=0	state=CHECKING_AC, alerts=[]	state=CHECKING_AC, alerts=[]	PASS
TC_M1_17	M1 Detection Engine	Fans and lights ON while room is occupied [EP (occupied)]	10 frames with people=4, fans_on=4, lights_on=6	state=OCCUPIED, alerts=[]	state=OCCUPIED, alerts=[]	PASS
TC_M1_18	M1 Detection Engine	Fan switched OFF and ON again during vacancy [Functional]	vacant: fans_on 2, 2, 0, 2	state=CHECKING_AC, alerts=[FAN_ON_VACANT, FAN_ON_VACANT]	state=CHECKING_AC, alerts=[FAN_ON_VACANT, FAN_ON_VACANT]	PASS
TC_M1_19	M1 Detection Engine	Mean temperature drop exactly 0.5 deg C [BVA (threshold)]	start 25.0 in all regions, after 60 s 24.5 in all regions	state=VACANT, ac_inferred_on=True, alerts=[AC_ON_VACANT]	state=VACANT, ac_inferred_on=True, alerts=[AC_ON_VACANT]	PASS
TC_M1_20	M1 Detection Engine	Mean temperature drop 0.49 deg C [BVA (threshold - 0.01)]	start 25.0, after 60 s 24.51 in all regions	state=VACANT, ac_inferred_on=False, alerts=[]	state=VACANT, ac_inferred_on=False, alerts=[]	PASS
TC_M1_21	M1 Detection Engine	Mean temperature drop 0.499 deg C [BVA (threshold - 0.001)]	start 25.0, after 60 s regions 24.501, 24.501, 24.501, 24.501	state=VACANT, ac_inferred_on=False, alerts=[]	state=VACANT, ac_inferred_on=False, alerts=[]	PASS
TC_M1_22	M1 Detection Engine	Different drop in each region (mean above threshold) [Functional]	start 25.0; after 60 s 24.0, 24.8, 24.2, 25.0 (mean drop 0.5)	state=VACANT, ac_inferred_on=True, alerts=[AC_ON_VACANT]	state=VACANT, ac_inferred_on=True, alerts=[AC_ON_VACANT]	PASS
TC_M1_23	M1 Detection Engine	Room warms up after vacancy (AC off) [EP (AC off)]	start 25.0, after 60 s 25.3 in all regions	state=VACANT, ac_inferred_on=False, alerts=[]	state=VACANT, ac_inferred_on=False, alerts=[]	PASS
TC_M1_24	M1 Detection Engine	Final reading 59 s after vacancy confirmed [BVA (wait - 1 s)]	drop 1.0 deg C at 59 s	state=CHECKING_AC, ac_inferred_on=None, alerts=[]	state=CHECKING_AC, ac_inferred_on=None, alerts=[]	PASS
TC_M1_25	M1 Detection Engine	Final reading exactly 60 s after vacancy confirmed [BVA (wait)]	drop 1.0 deg C at 60 s	state=VACANT, ac_inferred_on=True, alerts=[AC_ON_VACANT]	state=VACANT, ac_inferred_on=True, alerts=[AC_ON_VACANT]	PASS
TC_M1_26	M1 Detection Engine	Person enters during the AC check [Unit (path)]	vacancy at t=7, people=2 at t=30, empty again at t=67 with drop 1.0	state=OCCUPIED, ac_inferred_on=None, alerts=[]	state=OCCUPIED, ac_inferred_on=None, alerts=[]	PASS
TC_M1_27	M1 Detection Engine	vacancy_frames = 0 [BVA (min - 1)]	vacancy_frames=0, wait_seconds=120, delta_t_threshold=0.5	vacancy_frames must be between 1 and 300.	vacancy_frames must be between 1 and 300.	PASS
TC_M1_28	M1 Detection Engine	vacancy_frames = 1 and 300 [BVA (min / max)]	vacancy_frames=1, then 300	accepted / accepted	accepted / accepted	PASS

Test Case ID	Module	Test Scenario	Test Data	Expected Result	Actual Result	Status
TC_M1_29	M1 Detection Engine	vacancy_frames = 301 [BVA (max + 1)]	vacancy_frames=301	vacancy_frames must be between 1 and 300.	vacancy_frames must be between 1 and 300.	PASS
TC_M1_30	M1 Detection Engine	wait_seconds = 29 and 601 [BVA]	wait_seconds=29, then 601	wait_seconds must be between 30 and 600. / wait_seconds must be between 30 and 600.	wait_seconds must be between 30 and 600. / wait_seconds must be between 30 and 600.	PASS
TC_M1_31	M1 Detection Engine	delta_t_threshold = 0.09 and 5.1 [BVA]	delta_t_threshold=0.09, then 5.1	delta_t_threshold must be between 0.1 and 5.0. / delta_t_threshold must be between 0.1 and 5.0.	delta_t_threshold must be between 0.1 and 5.0. / delta_t_threshold must be between 0.1 and 5.0.	PASS
TC_M1_32	M1 Detection Engine	Non-numeric and fractional settings [EP (invalid)]	delta_t_threshold="abc"; vacancy_frames=10.5	delta_t_threshold must be a number. / vacancy_frames must be a whole number.	delta_t_threshold must be a number. / vacancy_frames must be a whole number.	PASS

M2 Dashboard & Control

Test Case ID	Module	Test Scenario	Test Data	Expected Result	Actual Result	Status
TC_M2_01	M2 Dashboard & Control	List monitored rooms [Functional]	GET /api/rooms	200: 2 rooms (CR-201, LAB-1)	200: 2 rooms (CR-201, LAB-1)	PASS
TC_M2_02	M2 Dashboard & Control	Post a valid reading [Functional / EP (valid)]	LAB-1, t=1, people=5, fans_on=4, lights_on=6, temps 25.0 x 4	200: state=OCCUPIED	200: state=OCCUPIED	PASS
TC_M2_03	M2 Dashboard & Control	Reading for a room that does not exist [EP (invalid)]	room_id=LAB-9	404: Room LAB-9 not found.	404: Room LAB-9 not found.	PASS
TC_M2_04	M2 Dashboard & Control	Reading without room_id [Validation]	body without room_id	400: room_id is required.	400: room_id is required.	PASS
TC_M2_05	M2 Dashboard & Control	Body is not valid JSON [Validation]	body = {people: 3	400: Request body must be valid JSON.	400: Request body must be valid JSON.	PASS
TC_M2_06	M2 Dashboard & Control	Region 3 temperature 50.5 deg C [Validation / BVA]	temps=[25, 25, 50.5, 25]	400: temperature of region 3 must be between 0 and 50 deg C.	400: temperature of region 3 must be between 0 and 50 deg C.	PASS
TC_M2_07	M2 Dashboard & Control	Acknowledge an open alert [Functional]	alert 1, operator='R. Kulkarni', remarks=''	200: Alert 1 acknowledged by R. Kulkarni.	200: Alert 1 acknowledged by R. Kulkarni.	PASS
TC_M2_08	M2 Dashboard & Control	Acknowledge the same alert twice [EP (invalid)]	alert 1 acknowledged, then again	409: Alert 1 is already acknowledged.	409: Alert 1 is already acknowledged.	PASS
TC_M2_09	M2 Dashboard & Control	Acknowledge an alert that does not exist [EP (invalid)]	alert 999	404: Alert 999 not found.	404: Alert 999 not found.	PASS
TC_M2_10	M2 Dashboard & Control	Operator name of 2 characters [BVA (min - 1)]	operator='Ra'	400: Operator name must be 3 to 40 characters (letters, spaces and dots only).	400: Operator name must be 3 to 40 characters (letters, spaces and dots only).	PASS
TC_M2_11	M2 Dashboard & Control	Operator name of 3 characters [BVA (min)]	operator='Ram'	200: Alert 1 acknowledged by Ram.	200: Alert 1 acknowledged by Ram.	PASS
TC_M2_12	M2 Dashboard & Control	Remarks of 201 characters [BVA (max + 1)]	remarks = 'A' x 201	400: Remarks cannot exceed 200 characters.	400: Remarks cannot exceed 200 characters.	PASS
TC_M2_13	M2 Dashboard & Control	Remarks of 200 characters [BVA (max)]	remarks = 'A' x 200	200: Alert 1 acknowledged by R. Kulkarni.	200: Alert 1 acknowledged by R. Kulkarni.	PASS
TC_M2_14	M2 Dashboard & Control	Manual override: switch fans OFF [Functional]	LAB-1, fan, OFF, 'R. Kulkarni', 'Room empty after lecture'	200: Command 1 queued: switch fan OFF in LAB-1.	200: Command 1 queued: switch fan OFF in LAB-1.	PASS

Test Case ID	Module	Test Scenario	Test Data	Expected Result	Actual Result	Status
TC_M2_15	M2 Dashboard & Control	Override for an unsupported appliance [EP (invalid)]	appliance='ac'	400: Appliance must be fan or light.	400: Appliance must be fan or light.	PASS
TC_M2_16	M2 Dashboard & Control	Action in lower case [EP (invalid)]	action='on'	400: Action must be ON or OFF.	400: Action must be ON or OFF.	PASS
TC_M2_17	M2 Dashboard & Control	Reason of 4 characters [BVA (min - 1)]	reason='Idle'	400: Reason must be 5 to 100 characters.	400: Reason must be 5 to 100 characters.	PASS
TC_M2_18	M2 Dashboard & Control	Reason of 5 characters [BVA (min)]	reason='Empty'	200: Command 1 queued: switch fan OFF in LAB-1.	200: Command 1 queued: switch fan OFF in LAB-1.	PASS
TC_M2_19	M2 Dashboard & Control	Reason of 100 characters [BVA (max)]	reason = 'R' x 100	200: Command 1 queued: switch fan OFF in LAB-1.	200: Command 1 queued: switch fan OFF in LAB-1.	PASS
TC_M2_20	M2 Dashboard & Control	Reason of 101 characters [BVA (max + 1)]	reason = 'R' x 101	400: Reason must be 5 to 100 characters.	400: Reason must be 5 to 100 characters.	PASS
TC_M2_21	M2 Dashboard & Control	ESP32 marks a command as done [Functional]	POST /api/commands/1/done	200: status=DONE	200: status=DONE	PASS
TC_M2_22	M2 Dashboard & Control	Command marked done twice [EP (invalid)]	POST /api/commands/1/done twice	409: Command 1 is already DONE.	409: Command 1 is already DONE.	PASS
TC_M2_23	M2 Dashboard & Control	Save valid detection settings [Functional]	vacancy_frames=8, wait_seconds=90, delta_t_threshold=0.8	200: Settings saved.	200: Settings saved.	PASS
TC_M2_24	M2 Dashboard & Control	Settings kept after server restart [Functional (database)]	save 8 / 90 / 0.8, restart server on the same database, GET /api/settings	vacancy_frames=8, wait_seconds=90, delta_t_threshold=0.8	vacancy_frames=8, wait_seconds=90, delta_t_threshold=0.8	PASS

Integration (M1 + M2)

Test Case ID	Module	Test Scenario	Test Data	Expected Result	Actual Result	Status
TC_INT_01	Integration	Engine alerts are stored and listed [Integration]	2 occupied frames, 5 vacant frames with 2 fans + 6 lights ON, drop 1.0 deg C after 60 s	AC_ON_VACANT, FAN_ON_VACANT, LIGHT_ON_VACANT	AC_ON_VACANT, FAN_ON_VACANT, LIGHT_ON_VACANT	PASS
TC_INT_02	Integration	Room status combines engine and database [Integration]	same readings as TC_INT_01; GET /api/rooms/LAB-1	state=VACANT, open_alerts=3, ac_inferred_on=True	state=VACANT, open_alerts=3, ac_inferred_on=True	PASS
TC_INT_03	Integration	Acknowledging an alert updates the room [Integration]	TC_INT_01 readings, acknowledge alert 1	open_alerts=2, alert 1 status=ACKNOWLEDGED	open_alerts=2, alert 1 status=ACKNOWLEDGED	PASS
TC_INT_04	Integration	Override reaches the ESP32 and stops alerts [Integration]	vacant with 2 fans ON; send fan OFF; ESP32 executes; 3 more frames with fans_on=0	pending=1, command status=DONE, FAN_ON_VACANT alerts=1	pending=1, command status=DONE, FAN_ON_VACANT alerts=1	PASS
TC_INT_05	Integration	New settings apply to the running engine [Integration]	vacancy_frames=3; 2 occupied + 3 empty frames	state=CHECKING_AC	state=CHECKING_AC	PASS
TC_INT_06	Integration	Every reading is stored [Integration (database)]	7 readings for LAB-1	readings stored=7	readings stored=7	PASS
TC_INT_07	Integration	Simulator scenario ac_left_on [Integration (end to end)]	simulator: people leave at 10 s, 2 fans + 6 lights + AC left on, 80 s	AC_ON_VACANT, FAN_ON_VACANT, LIGHT_ON_VACANT	AC_ON_VACANT, FAN_ON_VACANT, LIGHT_ON_VACANT	PASS

M2 Dashboard & Control (UI)

Test Case ID	Module	Test Scenario	Test Data	Expected Result	Actual Result	Status
TC_UI_01	M2 Dashboard & Control	Room that has sent no readings yet [Functional (UI)]	Open dashboard, room CR-201 (no readings)	NO DATA	NO DATA	PASS
TC_UI_02	M2 Dashboard & Control	Status card after the room becomes vacant [Functional (UI)]	Open room LAB-1	VACANT | people 0 | fans 2 | lights 6 | AC likely ON (mean drop 1.00 °C)	VACANT | people 0 | fans 2 | lights 6 | AC likely ON (mean drop 1.00 °C)	PASS
TC_UI_03	M2 Dashboard & Control	Heatmap shows the four region temperatures [Functional (UI)]	Open room LAB-1	24.0 °C, 24.0 °C, 24.0 °C, 24.0 °C	24.0 °C, 24.0 °C, 24.0 °C, 24.0 °C	PASS
TC_UI_04	M2 Dashboard & Control	Open alerts table [Functional (UI)]	Open room LAB-1	3 rows: AC ON VACANT, LIGHT ON VACANT, FAN ON VACANT	3 rows: AC ON VACANT, LIGHT ON VACANT, FAN ON VACANT	PASS
TC_UI_05	M2 Dashboard & Control	Acknowledge without selecting an alert [Validation (UI)]	Alert: -- Select --, operator 'R. Kulkarni'	Please select an alert.	Please select an alert.	PASS
TC_UI_06	M2 Dashboard & Control	Operator name of 1 character [Validation / BVA (UI)]	Alert #1, operator 'R'	Operator name must be 3 to 40 characters (letters, spaces and dots only).	Operator name must be 3 to 40 characters (letters, spaces and dots only).	PASS
TC_UI_07	M2 Dashboard & Control	Acknowledge alert #1 [Functional (UI)]	Alert #1, operator 'R. Kulkarni', remarks 'Fans switched off'	Alert 1 acknowledged by R. Kulkarni.	Alert 1 acknowledged by R. Kulkarni.	PASS
TC_UI_08	M2 Dashboard & Control	Acknowledged alert leaves the open list [Integration (UI)]	Reload dashboard, room LAB-1	2 rows; open alerts card shows 2	2 rows; open alerts card shows 2	PASS
TC_UI_09	M2 Dashboard & Control	Send manual override fan OFF [Functional (UI)]	Fan, Switch OFF, 'R. Kulkarni', 'Room empty after lecture'	Command 1 queued: switch fan OFF in LAB-1.	Command 1 queued: switch fan OFF in LAB-1.	PASS
TC_UI_10	M2 Dashboard & Control	Override without choosing an appliance [Validation (UI)]	Appliance: -- Select --, Switch OFF	Appliance must be fan or light.	Appliance must be fan or light.	PASS
TC_UI_11	M2 Dashboard & Control	Override without choosing ON or OFF [Validation (UI)]	Light, no action selected	Action must be ON or OFF.	Action must be ON or OFF.	PASS
TC_UI_12	M2 Dashboard & Control	Override reason of 3 characters [Validation / BVA (UI)]	Light, Switch OFF, reason 'abc'	Reason must be 5 to 100 characters.	Reason must be 5 to 100 characters.	PASS
TC_UI_13	M2 Dashboard & Control	Save valid settings [Functional (UI)]	frames 5, wait 60, threshold 0.5	Settings saved.	Settings saved.	PASS
TC_UI_14	M2 Dashboard & Control	Empty frames = 301 [Validation / BVA (UI)]	frames 301, wait 60, threshold 0.5	Empty frames to confirm vacancy must be between 1 and 300.	Empty frames to confirm vacancy must be between 1 and 300.	PASS
TC_UI_15	M2 Dashboard & Control	Threshold is not a number [Validation (UI)]	frames 5, wait 60, threshold 'abc'	Mean ΔT threshold must be a number.	Mean ΔT threshold must be a number.	PASS

Test execution summary

Module	Test cases	Run 1 passed	Run 1 failed	Run 2 (retest) passed	Run 2 (retest) failed
M1 Detection Engine	32	31	1	32	0
M2 Dashboard & Control	24	23	1	24	0
Integration (M1 + M2)	7	7	0	7	0
M2 Dashboard & Control (UI)	15	12	3	15	0
Total	**78**	**73**	**5**	**78**	**0**

9. Bugs Recorded in Jira

Every failure of run 1 was analysed and recorded as a Bug in Jira (SCEM Sprint 3). BUG-01 was first noticed while exploring the dashboard and is reproduced by TC_UI_01.

Bug ID	Bug description	Steps to reproduce	Severity / Priority	Status	Resolution
BUG-01 (SCEM-24)	Dashboard shows OCCUPIED for a room that has not sent any reading Expected: State badge shows NO DATA (FR-2.2) Actual: State badge shows OCCUPIED Found by: Exploratory testing of the dashboard; reproduced by TC_UI_01 in run1-initial	1. Start the server with a new database: python -m app.server --db data/new.db 2. Open http://127.0.0.1:8090 in the browser 3. Select room CR-201 (no reading has been posted for it)	Minor / Medium	Closed	Fixed - RoomMonitor.status() now reports NO_DATA until the first frame is received; the dashboard shows the NO DATA badge (app/engine.py). Retest: TC_UI_01 PASS in run2-retest
BUG-02 (SCEM-25)	False AC_ON_VACANT alert when the mean temperature drop is just below the threshold Expected: ac_inferred_on=False, no alert (mean drop 0.499 < 0.5, FR-1.6) Actual: ac_inferred_on=True and AC_ON_VACANT alert raised Found by: TC_M1_21 (BVA: threshold - 0.001) in run1-initial	1. Settings: vacancy_frames=5, wait_seconds=60, delta_t_threshold=0.5 2. Send 2 occupied frames, then 5 empty frames with all regions at 25.0 deg C 3. 60 s after vacancy is confirmed, send an empty frame with all regions at 24.501 deg C	Major / High	Closed	Fixed - the comparison uses the unrounded mean (rounded to 6 decimals only to remove floating-point noise); 2-3 decimal rounding is kept only for display (app/engine.py). Retest: TC_M1_19, TC_M1_20, TC_M1_21, TC_M1_22 PASS in run2-retest
BUG-03 (SCEM-26)	API returns 404 'Room None not found.' when room_id is missing Expected: 400: room_id is required. (FR-2.1) Actual: 404: Room None not found. Found by: TC_M2_04 in run1-initial	1. Start the server 2. POST /api/readings with a valid frame body that has no room_id field	Minor / Low	Closed	Fixed - a missing or empty room_id is rejected with 400 'room_id is required.' before the room lookup (app/server.py). Retest: TC_M2_04 PASS, TC_M2_03 still PASS in run2-retest
BUG-04 (SCEM-27)	Settings validation messages on the dashboard show internal field names Expected: Empty frames to confirm vacancy must be between 1 and 300. (FR-2.7) Actual: vacancy_frames must be between 1 and 300. Found by: TC_UI_14 and TC_UI_15 in run1-initial	1. Open the dashboard 2. In Detection settings enter Empty frames = 301, Wait = 60, Threshold = 0.5 3. Click Save settings	Trivial / Low	Closed	Fixed - the dashboard replaces the API field name with the form label before showing the message (app/static/app.js). Retest: TC_UI_14, TC_UI_15 PASS in run2-retest

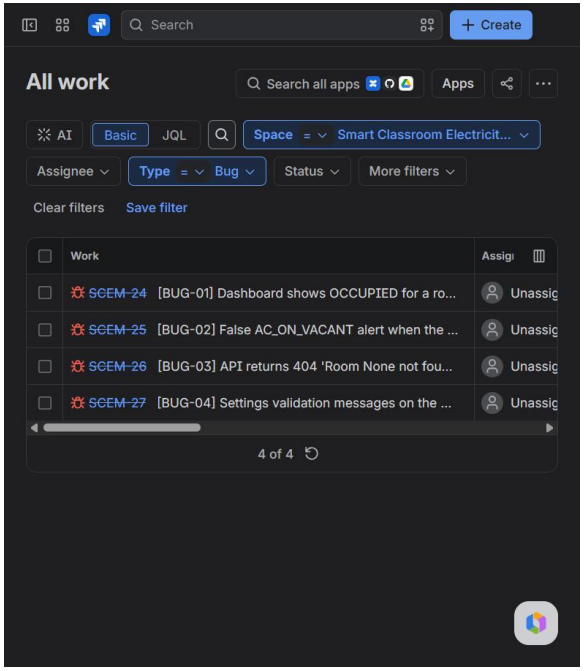


Figure 11: The four bugs in Jira (resolved keys are struck through)

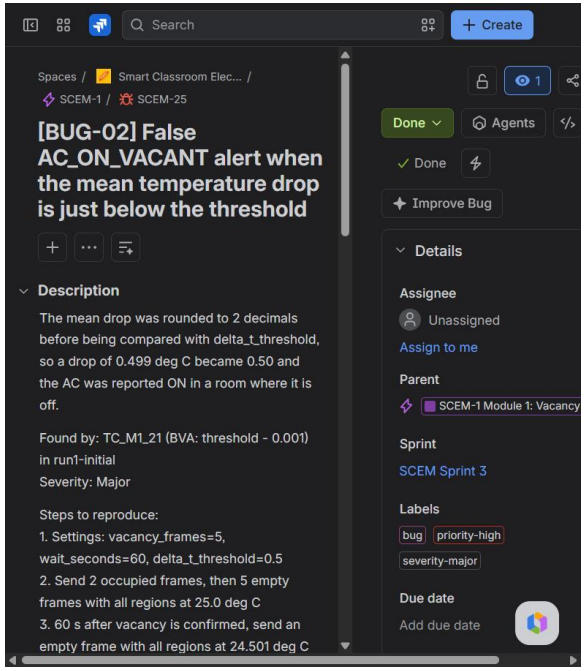


Figure 12: SCEM-25 (BUG-02): description, found by, severity, steps, sprint, labels

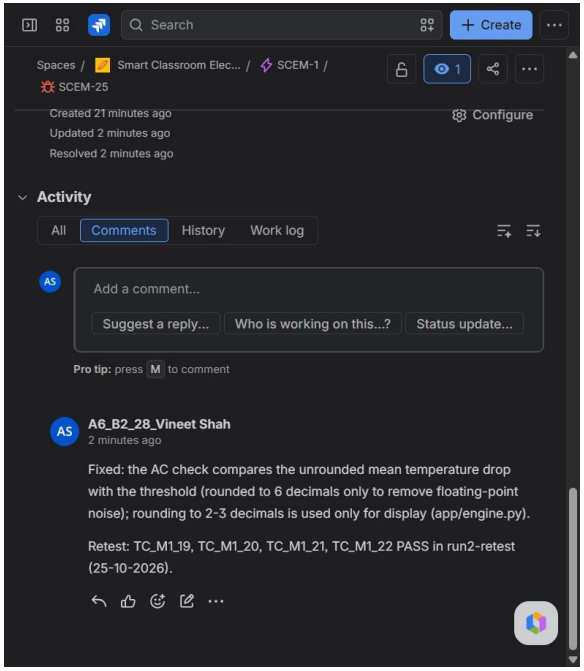


Figure 13: SCEM-25: fix and retest comment, resolved

10. Bug Fixing and Retesting Evidence

Test case	Bug	Run 1 (before fix) - actual	Run 1	Run 2 (after fix) - actual	Run 2
TC_UI_01	BUG-01	OCCUPIED	FAIL	NO DATA	PASS
TC_M1_21	BUG-02	state=VACANT, ac_inferred_on=True, alerts=[AC_ON_VACANT]	FAIL	state=VACANT, ac_inferred_on=False, alerts=[]	PASS
TC_M2_04	BUG-03	404: Room None not found.	FAIL	400: room_id is required.	PASS
TC_UI_14	BUG-04	vacancy_frames must be between 1 and 300.	FAIL	Empty frames to confirm vacancy must be between 1 and 300.	PASS
TC_UI_15	BUG-04	delta_t_threshold must be a number.	FAIL	Mean ΔT threshold must be a number.	PASS

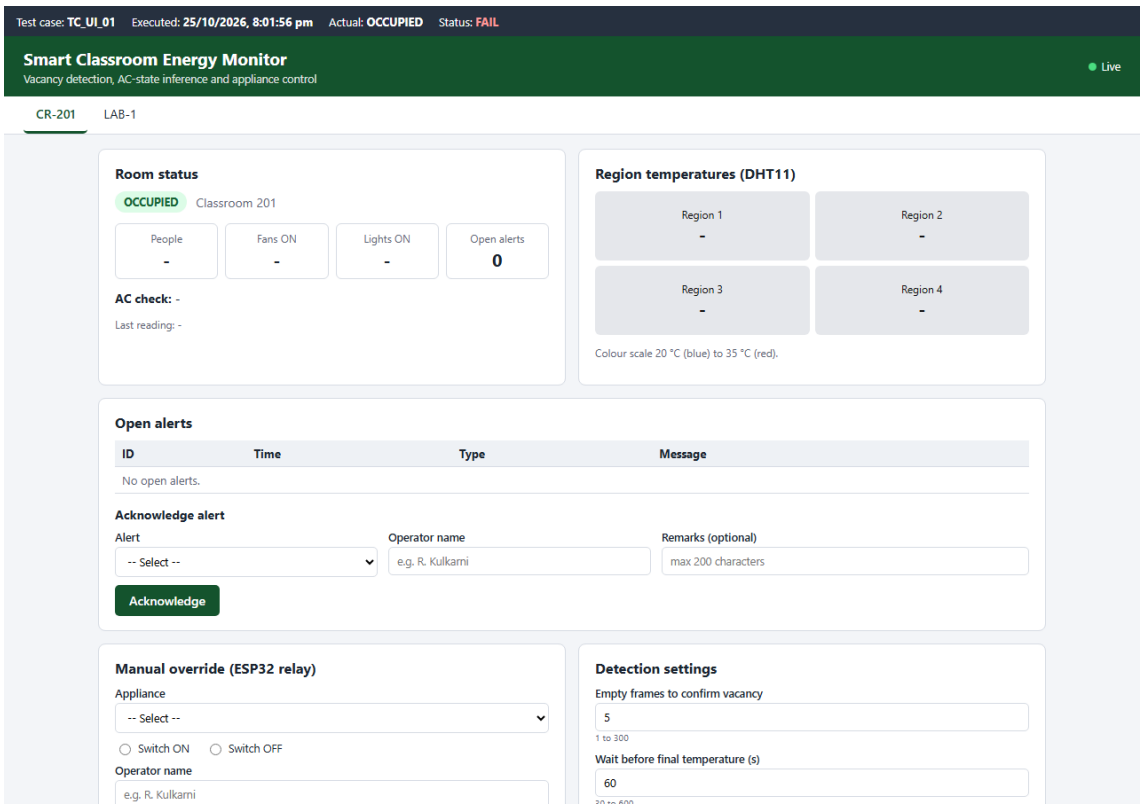


Figure 14: BUG-01 before the fix: CR-201 has no readings but is shown as OCCUPIED (TC_UI_01 FAIL)

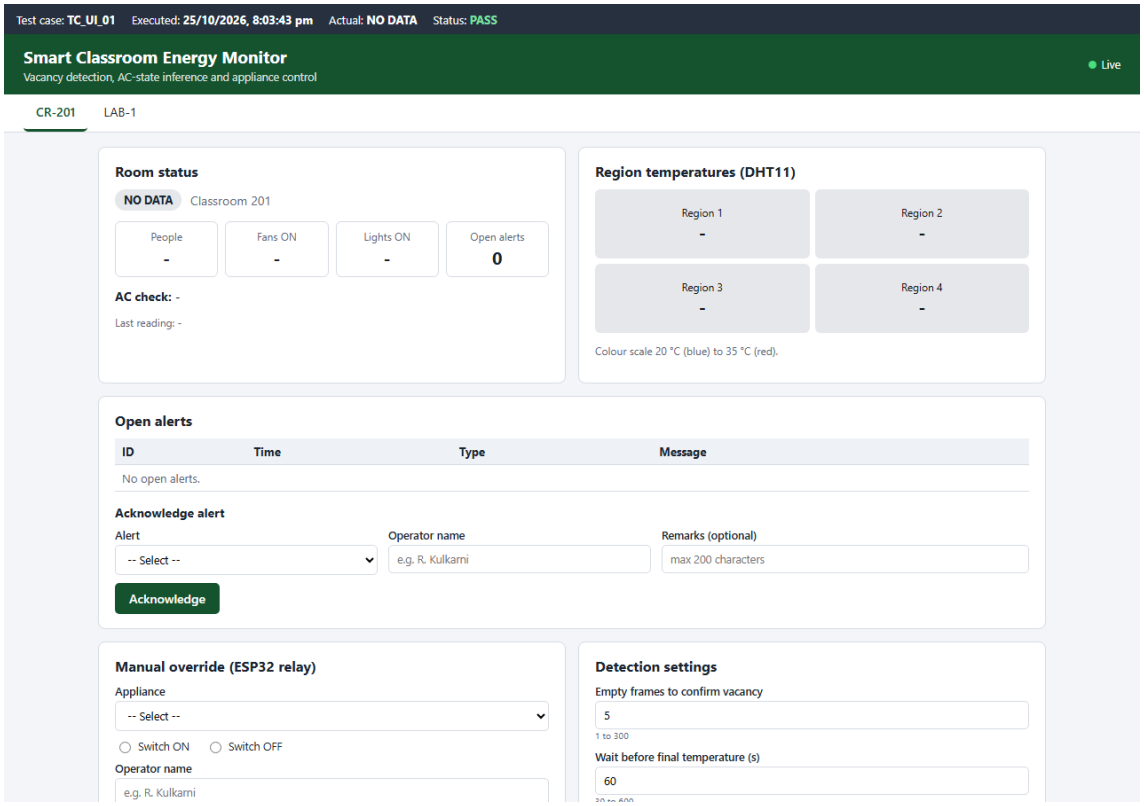


Figure 15: BUG-01 after the fix: NO DATA (TC_UI_01 PASS)

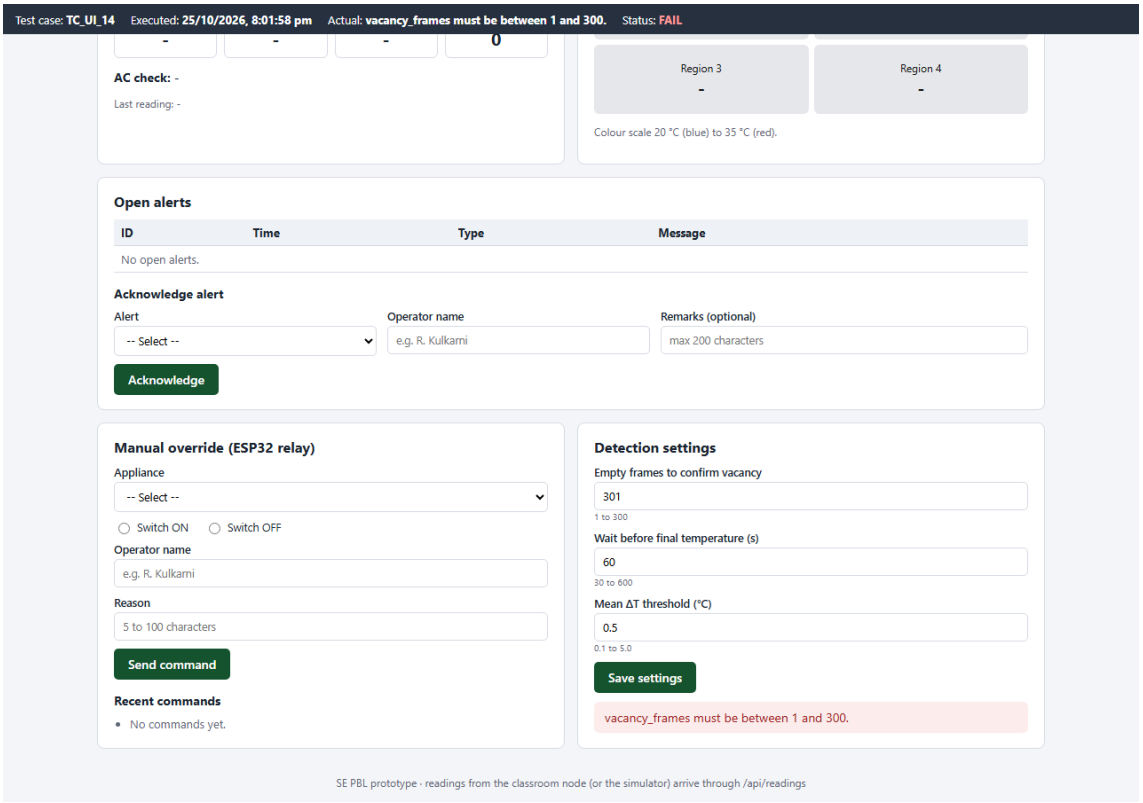


Figure 16: BUG-04 before the fix: internal field name in the message (TC_UI_14 FAIL)

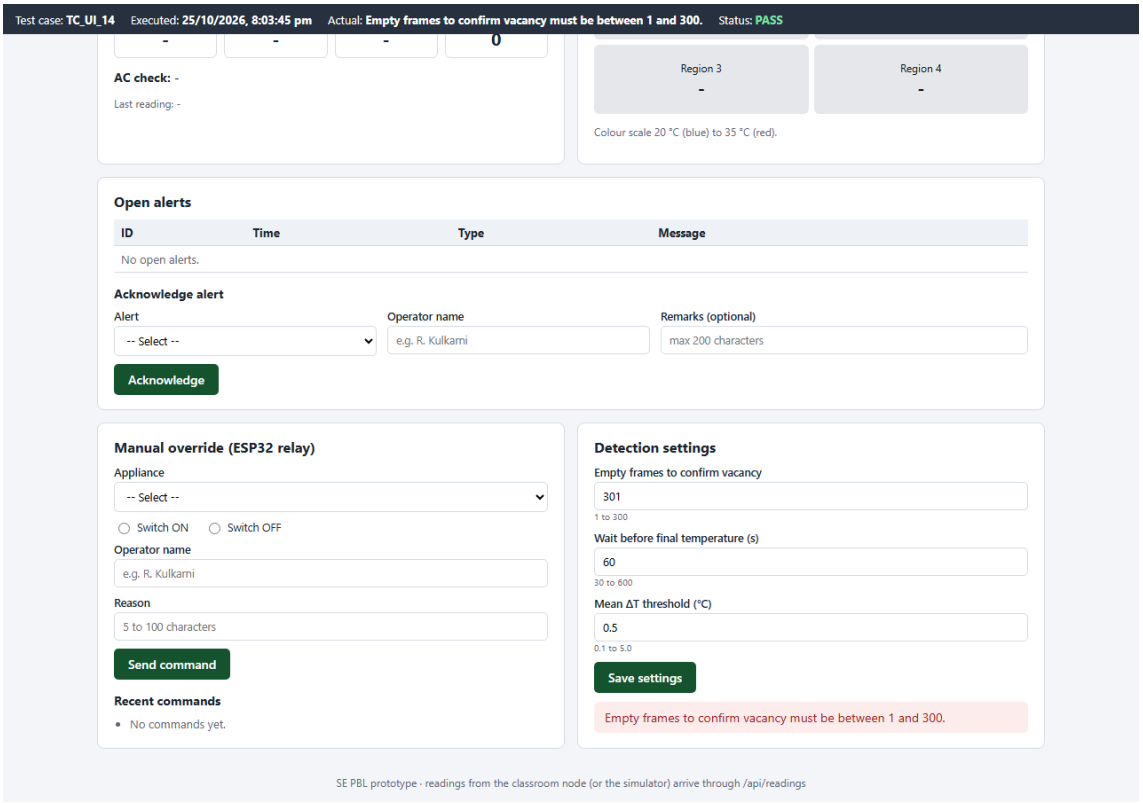


Figure 17: BUG-04 after the fix: message uses the form label (TC_UI_14 PASS)

Run 1 console output (failures only):

```
TC_M1_21   FAIL  Mean temperature drop 0.499 deg C
           expected: state=VACANT, ac_inferred_on=False, alerts=[]
           actual  : state=VACANT, ac_inferred_on=True, alerts=[AC_ON_VACANT]
TC_M2_04   FAIL  Reading without room_id
```

```
           expected: 400: room_id is required.
           actual  : 404: Room None not found.
TC_UI_01   FAIL  Room that has sent no readings yet
           expected: NO DATA
           actual  : OCCUPIED
TC_UI_14   FAIL  Empty frames = 301
           expected: Empty frames to confirm vacancy must be between 1 and 300.
           actual  : vacancy_frames must be between 1 and 300.
TC_UI_15   FAIL  Threshold is not a number
           expected: Mean ΔT threshold must be a number.
           actual  : delta_t_threshold must be a number.
78 test cases executed, 73 passed, 5 failed (23.9 s)
```

Run 2 console output (last lines):

```
TC_UI_12   PASS  Override reason of 3 characters
TC_UI_13   PASS  Save valid settings
TC_UI_14   PASS  Empty frames = 301
TC_UI_15   PASS  Threshold is not a number

78 test cases executed, 78 passed, 0 failed (21.5 s)
```

11. Final Working Demonstration

- Start the server: python -m app.server --port 8090 --db data/demo.db and open http://127.0.0.1:8090.
- Run the classroom simulator: python -m app.simulator --room LAB-1 --scenario ac_left_on.
- Watch LAB-1 change from OCCUPIED to CHECKING AC to VACANT; fan, light and AC alerts appear.
- Acknowledge an alert, send a fan OFF override, change a setting; run the all_off scenario for CR-201 to show the AC OFF result.
- Run python -m tests.run_tests to execute all 78 test cases.

Simulator log of the demonstration run (25-10-2026):

```
t=  0s  state=OCCUPIED people=12 fans=4 lights=6 temps=[24.6, 25.1, 24.2, 25.4]
t= 15s  state=CHECKING_AC people=0 fans=0 lights=0 temps=[24.62, 25.12, 24.23, 25.42]
t= 30s  state=CHECKING_AC people=0 fans=0 lights=0 temps=[24.68, 25.16, 24.3, 25.48]
t= 45s  state=CHECKING_AC people=0 fans=0 lights=0 temps=[24.74, 25.21, 24.38, 25.54]
t= 60s  state=CHECKING_AC people=0 fans=0 lights=0 temps=[24.8, 25.25, 24.45, 25.6]
t= 75s  state=VACANT people=0 fans=0 lights=0 temps=[24.86, 25.3, 24.53, 25.66]
-----
t=  0s  state=OCCUPIED people=12 fans=4 lights=6 temps=[24.6, 25.1, 24.2, 25.4]
t= 14s  ALERT FAN_ON_VACANT: 2 fan(s) ON in vacant room LAB-1.
t= 14s  ALERT LIGHT_ON_VACANT: 6 light(s) ON in vacant room LAB-1.
t= 15s  state=CHECKING_AC people=0 fans=2 lights=6 temps=[24.54, 25.05, 24.13, 25.35]
t= 30s  state=CHECKING_AC people=0 fans=2 lights=6 temps=[24.39, 24.93, 23.95, 25.21]
t= 45s  state=CHECKING_AC people=0 fans=2 lights=6 temps=[24.24, 24.81, 23.77, 25.08]
t= 60s  state=CHECKING_AC people=0 fans=2 lights=6 temps=[24.09, 24.69, 23.59, 24.94]
t= 74s  ALERT AC_ON_VACANT: AC appears to be ON in vacant room LAB-1 (mean temperature drop 0.58 deg C in 60 s).
t= 75s  state=VACANT people=0 fans=2 lights=6 temps=[23.94, 24.57, 23.41, 24.81]
```

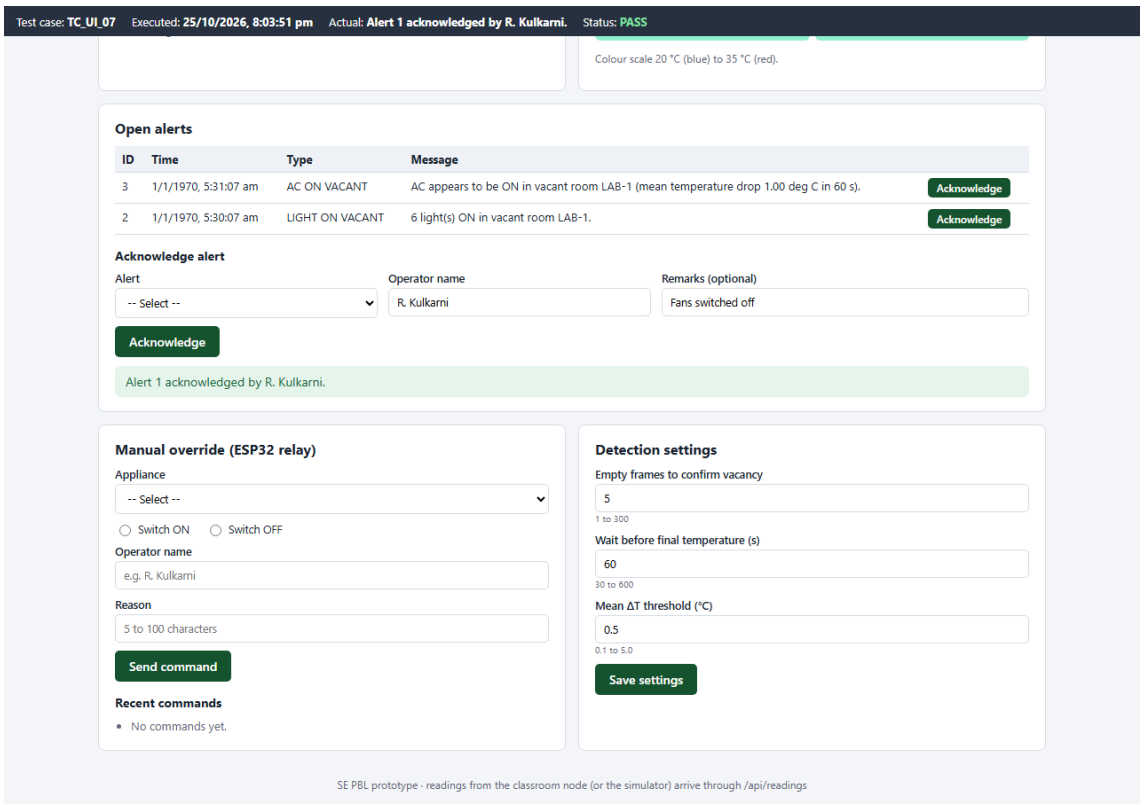


Figure 18: Alert acknowledged from the dashboard (TC_UI_07)

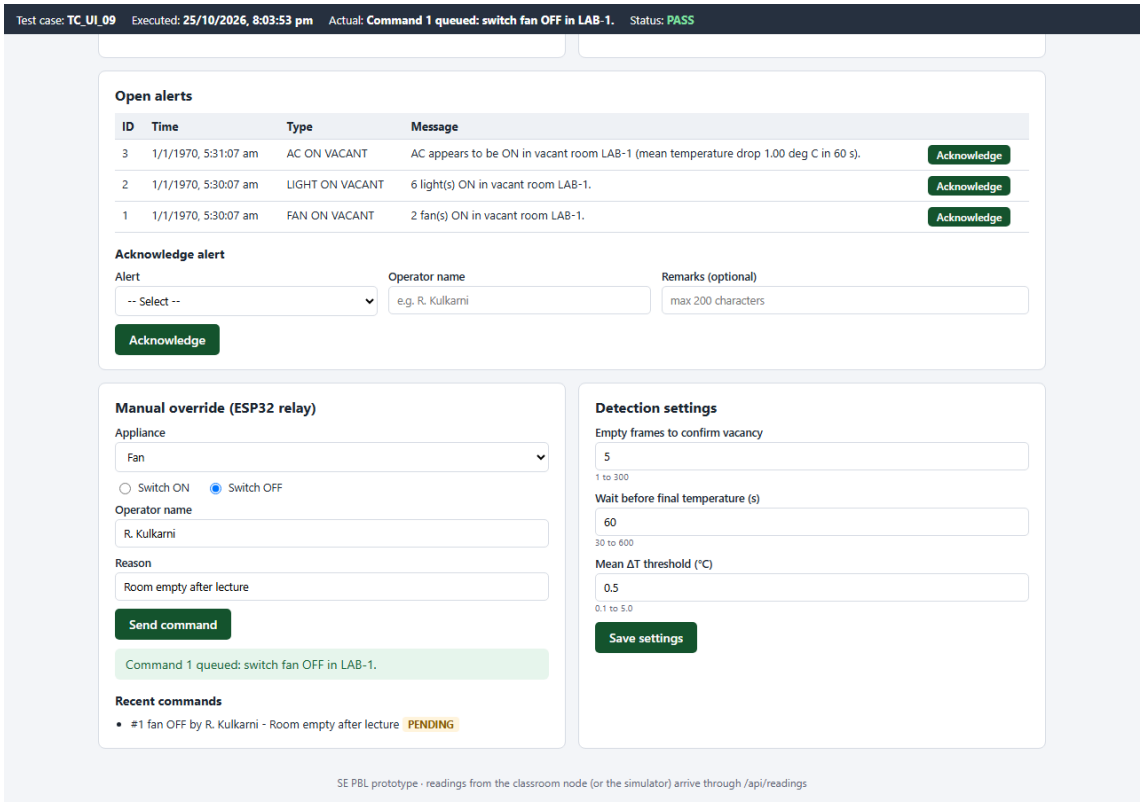


Figure 19: Manual override command queued (TC_UI_09)

12. Conclusion and Future Scope

Two modules of the Smart Classroom Electricity Management System were implemented individually: the vacancy and AC-state detection engine, and the monitoring dashboard with database, alerts, manual ESP32 override and settings. The work was planned and tracked in Jira as epics, user stories, tasks, sub-tasks and three sprints. 78 test cases using functional, unit, integration, boundary value, equivalence partitioning and validation testing were executed. The first run found 4 defects (5 failing test cases); all were recorded in Jira, fixed and retested, and the final run passed all 78 test cases.

- Connect the real CCTV + YOLOv8 detector, DHT11 sensors and ESP32 in place of the simulator.
- MQTT between the classroom node and the server, as in the original architecture.
- Daily energy report (hours of wasted appliance runtime per room) and mobile notifications.

Appendix A: app/engine.py (Module 1)

```python
"""Module 1: Vacancy and AC-state detection engine (edge decision engine).

Every frame from the classroom carries the YOLO detection counts (people, fans that are on, lights
that are on) and the temperatures of the four regions (one DHT11 sensor per region). The engine
confirms that the room is vacant over consecutive frames, raises an alert when a fan or light is left
on in a vacant room, and infers whether the AC is still running from the temperature drop (Delta T)
measured across the four regions after a waiting period.
"""
from dataclasses import dataclass

REGIONS = 4
TEMP_MIN, TEMP_MAX = 0.0, 50.0          # DHT11 measuring range in deg C
COUNT_MAX = 50                          # upper bound accepted for any detection count

OCCUPIED = "OCCUPIED"
CHECKING_AC = "CHECKING_AC"             # vacancy confirmed, waiting for the final temperatures
VACANT = "VACANT"                       # vacancy confirmed and AC check finished
NO_DATA = "NO_DATA"                     # reported until the room sends its first frame

FAN_ON_VACANT = "FAN_ON_VACANT"
LIGHT_ON_VACANT = "LIGHT_ON_VACANT"
AC_ON_VACANT = "AC_ON_VACANT"


class ValidationError(ValueError):
    """Raised when a frame or a setting is outside its allowed range."""


@dataclass
class Settings:
    vacancy_frames: int = 10            # consecutive empty frames needed to confirm vacancy
    wait_seconds: int = 120             # wait between the initial and final temperature readings
    delta_t_threshold: float = 0.5      # mean temperature drop (deg C) that means the AC is still on

    LIMITS = {"vacancy_frames": (1, 300), "wait_seconds": (30, 600), "delta_t_threshold": (0.1, 5.0)}

    @classmethod
    def from_dict(cls, data):
        """Builds validated settings; raises ValidationError naming the first bad field."""
        values = {}
        for name, (low, high) in cls.LIMITS.items():
            if name not in data:
                raise ValidationError("%s is required." % name)
            value = data[name]
            is_int_field = name != "delta_t_threshold"
            if isinstance(value, bool) or not isinstance(value, (int, float)):
                raise ValidationError("%s must be a number." % name)
            if is_int_field and value != int(value):
                raise ValidationError("%s must be a whole number." % name)
            if not low <= value <= high:
                raise ValidationError("%s must be between %s and %s." % (name, low, high))
            values[name] = int(value) if is_int_field else float(value)
        return cls(**values)

    def to_dict(self):
        return {"vacancy_frames": self.vacancy_frames, "wait_seconds": self.wait_seconds,
                "delta_t_threshold": self.delta_t_threshold}


@dataclass
class Frame:
    timestamp: float
    people: int
    fans_on: int
    lights_on: int
    temps: list

    @classmethod
    def from_dict(cls, data):
        """Validates one reading from the camera/sensor node."""
        ts = data.get("timestamp")
        if isinstance(ts, bool) or not isinstance(ts, (int, float)) or ts < 0:
            raise ValidationError("timestamp must be a non-negative number.")
        counts = {}
        for name in ("people", "fans_on", "lights_on"):
            value = data.get(name)
            if isinstance(value, bool) or not isinstance(value, int):
                raise ValidationError("%s must be a whole number." % name)
            if not 0 <= value <= COUNT_MAX:
                raise ValidationError("%s must be between 0 and %d." % (name, COUNT_MAX))
```

```
            counts[name] = value
        temps = data.get("temps")
        if not isinstance(temps, list) or len(temps) != REGIONS:
            raise ValidationError("temps must list exactly %d region temperatures." % REGIONS)
        for i, t in enumerate(temps):
            if isinstance(t, bool) or not isinstance(t, (int, float)):
                raise ValidationError("temperature of region %d must be a number." % (i + 1))
            if not TEMP_MIN <= t <= TEMP_MAX:
                raise ValidationError("temperature of region %d must be between 0 and 50 deg C." % (i + 1))
        return cls(float(ts), counts["people"], counts["fans_on"], counts["lights_on"], [float(t) for t in
temps])


class RoomMonitor:
    """Tracks one room frame by frame and returns the alerts each frame produces."""

    def __init__(self, room_id, settings=None):
        self.room_id = room_id
        self.settings = settings or Settings()
        self.state = OCCUPIED
        self.empty_frames = 0
        self.last_timestamp = None
        self.initial_temps = None
        self.check_started = None
        self.fan_alerted = False
        self.light_alerted = False
        self.last_mean_drop = None
        self.ac_inferred_on = None
        self.last_frame = None

    def process(self, frame):
        if self.last_timestamp is not None and frame.timestamp <= self.last_timestamp:
            raise ValidationError("timestamp must be later than the previous frame.")
        self.last_timestamp = frame.timestamp
        self.last_frame = frame
        alerts = []

        if frame.people > 0:
            self._reset_to_occupied()
            return alerts

        self.empty_frames += 1
        if self.state == OCCUPIED and self.empty_frames >= self.settings.vacancy_frames:
            self.state = CHECKING_AC
            self.initial_temps = list(frame.temps)
            self.check_started = frame.timestamp

        if self.state == OCCUPIED:
            return alerts

        alerts += self._appliance_alerts(frame)

        if self.state == CHECKING_AC and frame.timestamp - self.check_started >= self.settings.wait_seconds:
            drops = [start - end for start, end in zip(self.initial_temps, frame.temps)]
            # Compare at full precision (6 decimals only removes float noise); rounding to 2 decimals
            # before comparing turned a 0.499 deg C drop into 0.50 and raised a false AC alert (BUG-02).
            mean_drop = round(sum(drops) / REGIONS, 6)
            self.last_mean_drop = round(mean_drop, 3)
            self.ac_inferred_on = mean_drop >= self.settings.delta_t_threshold
            self.state = VACANT
            if self.ac_inferred_on:
                alerts.append(self._alert(AC_ON_VACANT, frame.timestamp,
                                          "AC appears to be ON in vacant room %s (mean temperature drop %.2f
deg C "
                                          "in %d s)." % (self.room_id, self.last_mean_drop,
self.settings.wait_seconds),
                                          self.last_mean_drop))
        return alerts

    def _appliance_alerts(self, frame):
        alerts = []
        if frame.fans_on > 0 and not self.fan_alerted:
            alerts.append(self._alert(FAN_ON_VACANT, frame.timestamp,
                                      "%d fan(s) ON in vacant room %s." % (frame.fans_on, self.room_id),
frame.fans_on))
        if frame.lights_on > 0 and not self.light_alerted:
            alerts.append(self._alert(LIGHT_ON_VACANT, frame.timestamp,
                                      "%d light(s) ON in vacant room %s." % (frame.lights_on, self.room_id),
                                      frame.lights_on))
        # An appliance switched off and on again during the same vacancy is reported again.
        self.fan_alerted = frame.fans_on > 0
        self.light_alerted = frame.lights_on > 0
```

```
159          return alerts
160
161      def _reset_to_occupied(self):
162          self.state = OCCUPIED
163          self.empty_frames = 0
164          self.initial_temps = None
165          self.check_started = None
166          self.fan_alerted = False
167          self.light_alerted = False
168
169      def _alert(self, kind, timestamp, message, value):
170          return {"room_id": self.room_id, "type": kind, "timestamp": timestamp, "message": message, "value":
value}
171
172      def status(self):
173          f = self.last_frame
174          return {
175              "room_id": self.room_id,
176              "state": self.state if f else NO_DATA,
177              "empty_frames": self.empty_frames,
178              "people": f.people if f else None,
179              "fans_on": f.fans_on if f else None,
180              "lights_on": f.lights_on if f else None,
181              "temps": f.temps if f else None,
182              "last_timestamp": self.last_timestamp,
183              "ac_check_seconds_left": (max(0, round(self.settings.wait_seconds - (self.last_timestamp -
self.check_started)))
184                                        if self.state == CHECKING_AC else None),
185              "mean_temp_drop": self.last_mean_drop,
186              "ac_inferred_on": self.ac_inferred_on,
187          }
```